



**Instituto Universitario de Ciencias de la Educación
Universidad de Salamanca**

Documentación del proyecto

**Web institucional del IUCE
Anexo B: Especificación de Diseño**

Elaborado por
Enrique González Gutiérrez

Salamanca, septiembre de 2026

Índice

Especificación de diseño.....	3
B.1. Introducción.....	3
B.1.1. Visión general del stack.....	3
B.2. Diseño de datos.....	4
B.2.1. Enumeraciones.....	4
B.2.2. Entidades de contenido.....	4
B.2.3. Entidades de soporte.....	5
B.2.4. Entidades del área de miembros.....	5
B.2.5. Decisiones de diseño de datos.....	6
B.3. Diseño procedimental.....	7
B.3.1. Publicación y traducción de una noticia.....	7
B.3.2. Acceso al área de miembros (magic link).....	7
B.3.3. Resolución de una pieza de contenido editable.....	8
B.3.4. Formulario de contacto.....	8
B.3.5. Ciclo de vida de la noticia.....	8
B.3.6. Últimos artículos de la dirección (ORCID).....	9
B.4. Diseño arquitectónico.....	9
B.4.1. Arquitectura en capas.....	9
B.4.2. Mapa de rutas.....	10
B.4.3. Arquitectura de la versión inglesa.....	11
B.4.4. Presentación y experiencia de usuario.....	11
B.4.5. Seguridad.....	12
B.4.6. Rendimiento.....	12
B.4.7. Despliegue.....	13

Anexo B

Especificación de diseño

B.1. Introducción

Este anexo describe cómo está construido el sistema en tres planos complementarios: el diseño de datos (entidades y relaciones), el diseño procedimental (los flujos que articulan el funcionamiento) y el diseño arquitectónico (capas, rutas, internacionalización, seguridad y despliegue). Mientras que el Anexo A establece qué debe hacer el sistema y por qué, este anexo documenta cómo se ha resuelto, tal y como está implementado.

B.1.1. Visión general del stack

Capa	Tecnología
Frontend	Next.js 14 (App Router) + TypeScript 5 · React 18 (Server Components por defecto)
Estilos / UI	Tailwind CSS 3 · componentes propios · tokens de marca como variables CSS (tema claro/oscuro)
Backend / API	Route Handlers de Next.js (REST) + Prisma 6 (ORM)
Base de datos	PostgreSQL 16
Autenticación	NextAuth.js v5 con dos proveedores de credenciales: panel (email + contraseña bcrypt) y área de miembros (magic link)
Editor	TipTap 3 (HTML como formato de almacenamiento)
Email	Resend, con plantillas HTML propias y logo incrustado
Publicaciones	API pública de ORCID: último artículo de cada miembro de la dirección, con caché de 24 horas y lista editable de reserva
Gráficas	Recharts (página de Estadísticas)
Imágenes	sharp (fotos de miembros a 512 px, imagen OpenGraph)
Despliegue	Desarrollo: Docker (solo PostgreSQL, puerto 5433) + servidor de desarrollo · Producción: Docker (app + PostgreSQL) tras Apache 2 con Let's Encrypt en el CPD-USAL

Tabla 1. Visión general del stack.

Todo el sistema se escribe en un único lenguaje (TypeScript) de extremo a extremo: el esquema de datos, las consultas, la lógica de servidor y la interfaz. Prisma genera un cliente tipado a partir de `prisma/schema.prisma`, que actúa como fuente única de verdad del modelo de datos. No se emplea un paradigma orientado a objetos clásico, sino la arquitectura funcional y modular propia de Next.js 14: la lógica reside en funciones puras y servicios (`src/lib`), en componentes React y en manejadores de ruta; la vista estructural del sistema queda mejor representada por el modelo de datos y el mapa de rutas que por un diagrama de clases.

B.2. Diseño de datos

El esquema completo vive en prisma/schema.prisma y se sincroniza con la base de datos mediante prisma db push. Se describen a continuación las enumeraciones y las entidades, agrupadas por área.

B.2.1. Enumeraciones

Enumeración	Valores	Uso
Role	ADMIN · SUPER_ADMIN	Rol de las cuentas del panel (User.role).
NewsStatus	DRAFT · PUBLISHED · ARCHIVED	Ciclo de vida de una noticia.
EventStatus	UPCOMING · PAST · CANCELLED	Estado de un evento.

Tabla 2. Enumeraciones del esquema.

B.2.2. Entidades de contenido

Campo	Descripción
id, slug	Identificador interno y slug único de la URL pública.
title / titleEn	Título en español y, opcionalmente, en inglés (patrón repetido en excerpt/excerptEn y content/contentEn).
content	Cuerpo en HTML, tal y como lo produce el editor TipTap.
coverImage	URL de la imagen de portada (opcional; sin ella, la web pinta una portada de marca).
category	Categoría editorial (taxonomía heredada y ampliada en la migración).
status	DRAFT · PUBLISHED · ARCHIVED.
internal	Noticia interna: solo visible en el área de miembros, nunca en la web pública.
publishedAt, createdAt, updatedAt	Fechas de publicación y auditoría.

Tabla 3. Entidad News (noticias).

Campo	Descripción
pageSlug	Página a la que pertenece la pieza (instituto, formacion, estadísticas...).
blockKey	Clave de la pieza dentro de la página. Convenciones: el sufijo «:en» guarda la versión inglesa de la misma pieza; el prefijo «list:» indica una lista estructurada cuyo contenido es JSON.
content	Texto (o JSON, en las listas) que prevalece sobre el valor de fábrica del registro en código.
(único)	Restricción de unicidad sobre el par (pageSlug, blockKey).

Tabla 4. Entidad *ContentBlock* (piezas editables de páginas estáticas).

Campo	Descripción
name, area, role	Nombre, área de investigación y cargo (Directora, Subdirector, Secretario; vacío = miembro).
email, extension	Correo institucional (con botón de copiar en la web; da acceso al área de miembros) y extensión telefónica.
photo	Fotografía (subida y redimensionada a 512 px, elegida de la biblioteca o URL).
portalUrl, orcid, scopus	Enlaces a su perfil del Portal de la Investigación, ORCID y Scopus.
active, order	Visibilidad en la web y orden de aparición.
groupId → ResearchGroup	Grupo de investigación al que pertenece (pinta su distintivo en la ficha).

Tabla 5. Entidad *Member* (miembros del Instituto).

Entidad	Campos y notas
ResearchGroup	acronym, name/nameEn, lead (responsable), url, logo, chip (distintivo, p. ej. «UIC 081») y relación 1–N con Member. Los nueve grupos oficiales se siembran desde código y son editables.
Project	title, funder, ip, line, scope (ámbito), amount y period (textos originales de la memoria de acreditación), startYear/endYear, active (visible) e iuceLed («proyecto del IUCE»: la web pública solo lista los que lo tienen activado).
Event	title/titleEn, type (Congreso, Seminario, Jornada), startsAt/endsAt, location, url, image y status. En la web pública, «próximo» o «celebrado» se deriva de las fechas; el campo status solo se aplica manualmente para CANCELLED.

Tabla 6. Entidades *ResearchGroup*, *Project* y *Event*.

B.2.3. Entidades de soporte

Entidad	Uso
User	Cuentas del panel: email único, nombre, passwordHash (bcrypt) y rol ADMIN o SUPER_ADMIN.
FileAsset	Biblioteca de archivos subidos desde el panel (nombre, tipo MIME, tamaño, URL bajo /uploads).
ContactMessage	Mensajes del formulario de contacto, con estado NEW/REPLIED.
PageVisibility	Visibilidad de páginas públicas: el slug de la página y su marca hidden. Sin fila, la página se ve.
PageView	Analítica ligera de visitas (ruta, referencia y visitante anónimo por día).

Tabla 7. Entidades de soporte.

B.2.4. Entidades del área de miembros

Entidad	Uso
IntranetUser	Cuentas del área privada: correo único, nombre, activa/vetada y último

Entidad	Uso
	acceso. Se crean desde el panel o automáticamente en el primer acceso de un miembro con correo en su ficha.
IntranetToken	Token del enlace de acceso: identificador (correo) + token único con caducidad (30 minutos); se consume con el primer uso.
IntranetDocument	Documentos internos: título, descripción y fichero almacenado fuera de public/ (storage/intranet), servido solo con sesión a través de la API.

Tabla 8. Entidades del área de miembros.

La figura siguiente reúne la vista lógica completa: las entidades de contenido (con la única relación del modelo, la pertenencia del miembro a su grupo), las de soporte y las del área de miembros.

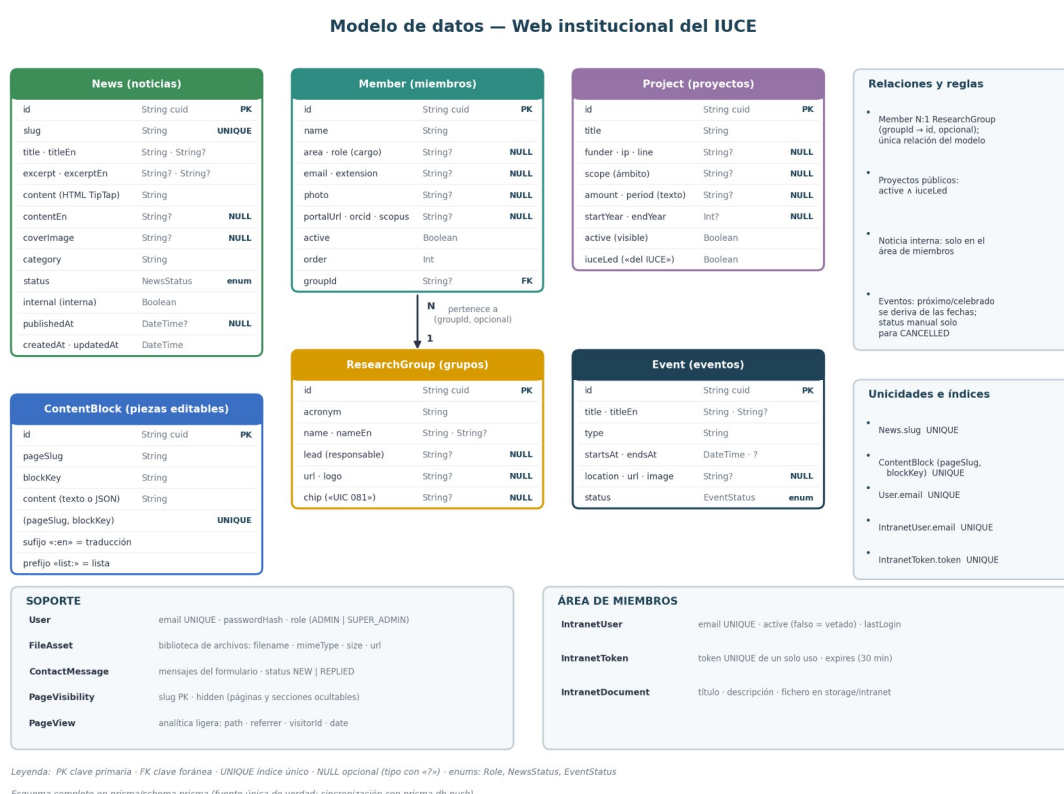


Figura 1. Modelo de datos: entidades y campos, relación Member–ResearchGroup, reglas y unicidades.

B.2.5. Decisiones de diseño de datos

- **HTML como formato de contenido.** El cuerpo de las noticias y los bloques largos almacenan el HTML que produce TipTap, el mismo formato heredado de WordPress en la migración: permite conservar el histórico intacto y editar cualquier noticia, antigua o nueva, con el mismo editor.
- **Un almacén genérico de piezas editables.** En lugar de una tabla por página, ContentBlock guarda cualquier pieza identificada por (página, clave), con dos convenciones en la clave: «:en» para la traducción y «:list» para listas cuyo contenido

es JSON con elementos tipados. Los valores de fábrica viven en registros TypeScript versionados (page-blocks.ts, list-blocks.ts y sus equivalentes -en), de modo que la base de datos solo contiene lo editado y siempre existe un texto de reserva.

- **Campos *En paralelos en lugar de tablas de traducción.** Para dos idiomas fijos, los campos paralelos (titleEn, contentEn, nameEn, titleEn) son más simples que un modelo de traducciones N-idiomas, consultan sin joins y mantienen el fallback trivial.
- **Textos originales en Proyectos.** Los importes y periodos de los proyectos se guardan como texto original de la memoria de acreditación (p. ej. «60.500,00 €»), evitando pérdidas de matiz en importes plurianuales; los años numéricos (startYear/endYear) existen aparte para filtrar.
- **Esquema sincronizado con db push y semillas idempotentes.** El proyecto usa prisma db push (sin ficheros de migración) más una semilla y scripts de carga re-ejecutables que nunca pisan lo editado desde el panel; reconstruir una base de datos desde cero es siempre posible.

B.3. Diseño procedimental

B.3.1. Publicación y traducción de una noticia

El guardado de una noticia desde el panel sigue esta secuencia: (1) validación de los campos con esquemas Zod compartidos; (2) generación del slug si es nueva; (3) persistencia con Prisma; (4) si hay clave de DeepL configurada, traducción automática de título, extracto y cuerpo hacia los campos *En (los editores pueden retocar el resultado desde el propio panel); (5) al publicarse, la noticia entra en el listado, el RSS y el sitemap, y su ruta /en/noticias/... sirve la versión inglesa con fallback al español si aún no existe traducción.

B.3.2. Acceso al área de miembros (magic link)

La secuencia completa del acceso sin contraseña:

- 1. El usuario introduce su correo en /miembros; la petición pasa por la limitación de tasa.
- 2. La regla de acceso decide (lib/intranet-access.ts): una cuenta desactivada veta siempre; en ausencia de cuenta, el correo de una ficha de miembro activa da acceso (con alta automática); las direcciones autorizadas a mano entran igualmente.
- 3. Se genera un token de un solo uso con caducidad de 30 minutos (IntranetToken) y se envía por correo con la plantilla institucional.
- 4. El enlace vuelve a la aplicación, que consume el token (se borra al usarse) e inicia sesión mediante el proveedor «intranet» de NextAuth; la sesión JWT lleva el rol INTRANET.
- 5. El rol INTRANET da acceso al área y a sus ficheros, pero nunca al panel de administración; los roles de administración, a la inversa, también pueden entrar en el área.

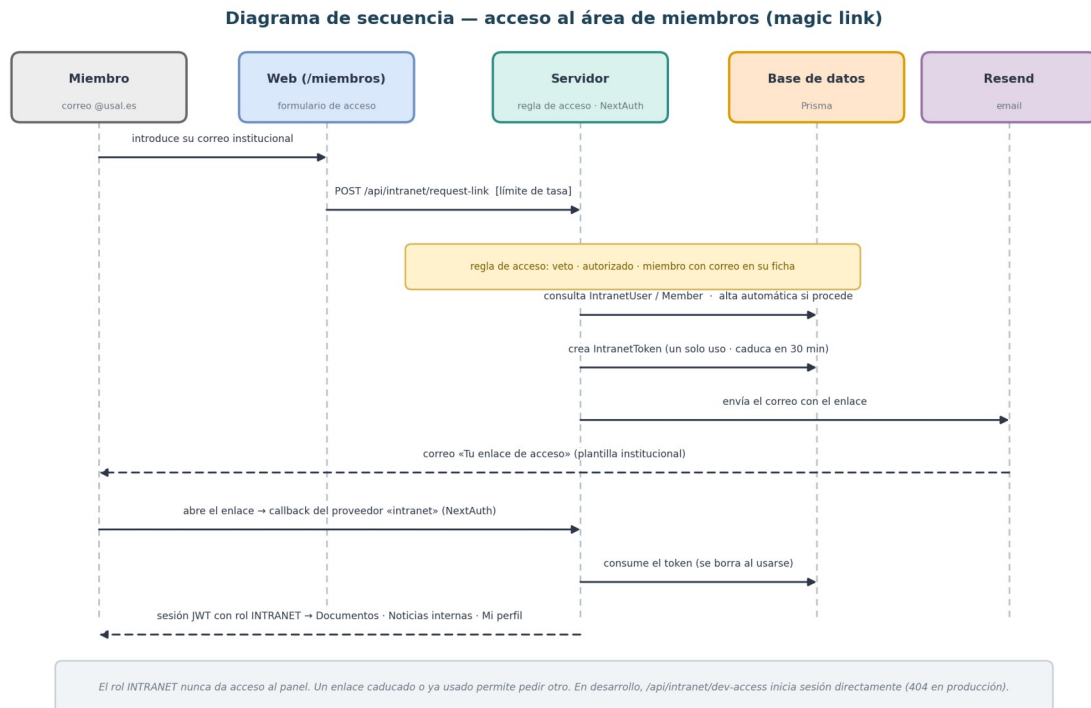


Figura 2. *Diagrama de secuencia del acceso al área de miembros.*

B.3.3. Resolución de una pieza de contenido editable

Cuando una página pública pide un bloque o una lista, el servicio resuelve el texto en cascada: (1) fila editada en base de datos —con clave «:en» si la petición llega en inglés—; (2) registro estático del idioma correspondiente; (3) registro español como última reserva. El resultado práctico: lo editado prevalece, lo no editado siempre tiene texto, y la ruta /en jamás rompe aunque falte una traducción.

B.3.4. Formulario de contacto

El envío valida los campos (Zod), aplica limitación de tasa, guarda el mensaje (ContactMessage), notifica por correo a la administración con responder-a apuntando al remitente y devuelve un acuse de recibo automático con la plantilla institucional. Los errores del proveedor de correo se comprueban explícitamente: si el envío falla, el usuario no recibe un falso «enviado».

B.3.5. Ciclo de vida de la noticia

Una noticia nace como borrador (DRAFT), pasa a publicada (PUBLISHED) cuando se aprueba —momento en el que se fija su fecha de publicación si no la tenía— y puede archivarse (ARCHIVED) para retirarla del sitio sin borrarla. La marca interna es ortogonal al estado: una noticia interna publicada es visible solo en el área de miembros.

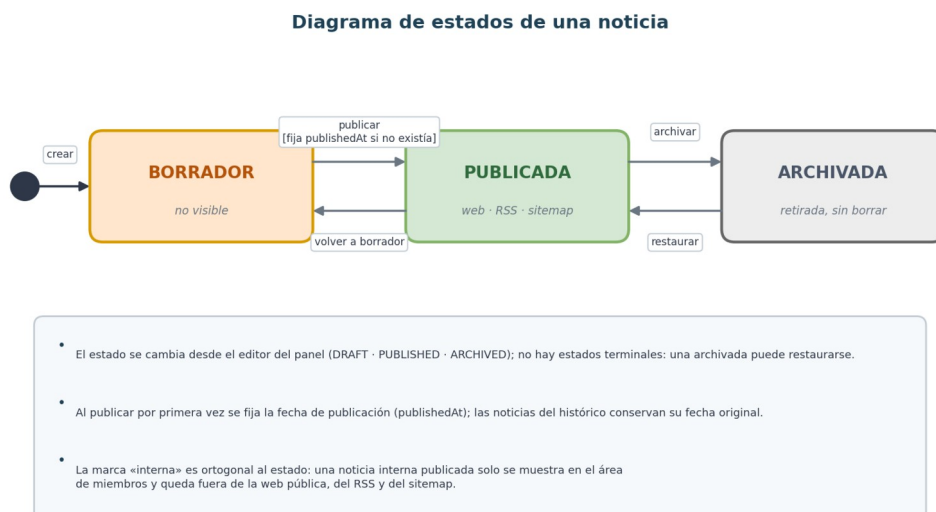


Figura 3. *Diagrama de estados de una noticia (transiciones y reglas).*

B.3.6. Últimos artículos de la dirección (ORCID)

La banda de publicaciones de /investigacion se alimenta de la API pública de ORCID (pub.orcid.org, sin clave): para la directora, el subdirector y el secretario académico se consulta su lista de obras, se toma la más reciente por fecha de publicación y se completa con revista, autores y enlace (DOI). Las respuestas se cachean 24 horas con el revalidate de fetch, de modo que la banda se renueva sola cuando la dirección publica algo nuevo, sin cron ni edición manual. Si ORCID no responde o algún perfil no devuelve resultados, la página cae a la lista editable del panel, que se conserva como reserva.

B.4. Diseño arquitectónico

B.4.1. Arquitectura en capas

La aplicación se organiza en cuatro capas dentro de un único proyecto: (1) la capa de presentación, con páginas y componentes React renderizados en servidor por defecto (los componentes de cliente se reservan para interactividad: menú, editor, gráficas, formularios); (2) la capa de API, con Route Handlers REST bajo /api para las operaciones del panel, el contacto y el área de miembros; (3) la capa de servicios (src/lib), donde reside la lógica de negocio y de acceso a datos (servicios de noticias, contenido, proyectos, visibilidad, acceso al área, correo, validaciones, límites de tasa); y (4) la capa de datos, el cliente Prisma sobre PostgreSQL. El middleware, en el borde, resuelve idioma y autorización antes de que la petición llegue a las capas anteriores.

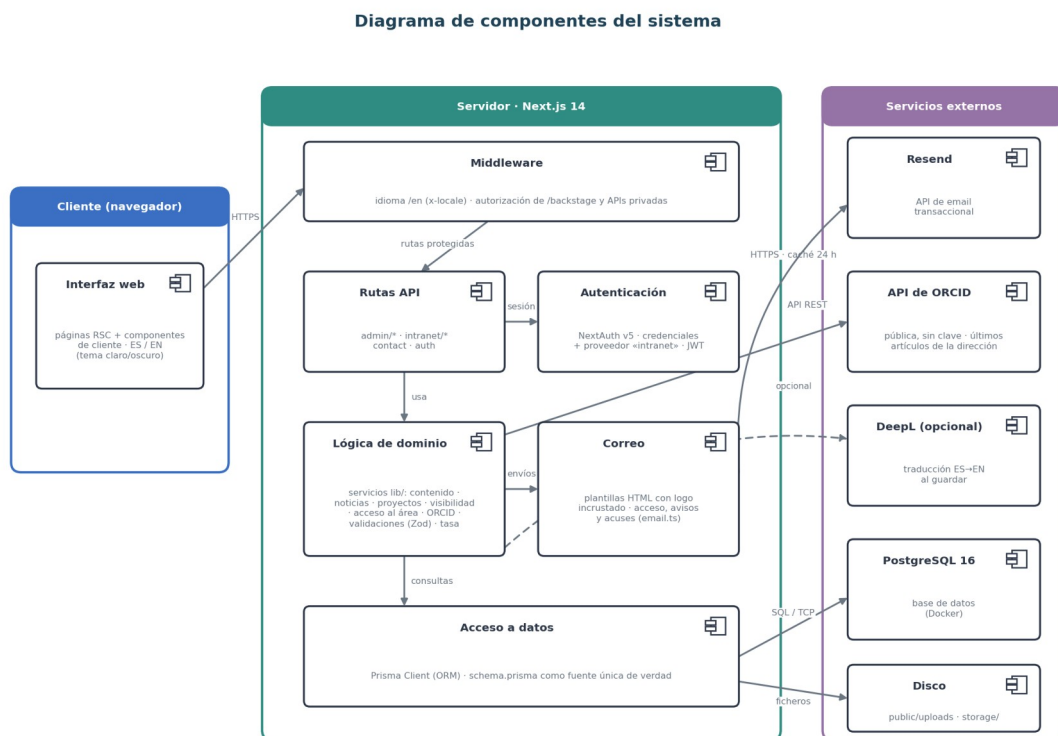


Figura 4. Diagrama de componentes del sistema (cliente, servidor Next.js y servicios externos).

B.4.2. Mapa de rutas

Zona	Rutas
Sitio público	/ (portada) · /instituto · /investigacion · /transferencia · /formacion · /doctorado · /estadisticas · /eventos · /seminario-iuce · /noticias y /noticias/[slug] · /contacto · /aviso-legal · /accesibilidad · /politica-de-cookies
Versión inglesa	/en y /en/*: espejo de todo el sitio público (reescritura interna a la ruta española con cabecera de idioma).
Área de miembros	/miembros (acceso y portada del área) · /miembros/noticias y /miembros/noticias/[slug] · /miembros/perfil. Las rutas antiguas /intranet/* redirigen (308) conservando la query.
Autenticación	/auth/signin (login del panel) · /api/auth/[...nextauth]
Panel (/backstage)	/backstage (dashboard) · news (+new, [id]) · pages (bloques y listas) · visualizacion · members · groups · events · projects · files · messages · intranet (+users, files) · settings
API pública	/api/contact · /api/intranet/request-link ·

Zona	Rutas
	/api/intranet/files/[id] · /api/intranet/profile (+photo) · /feed.xml · sitemap.xml, robots.txt, imagen OpenGraph
API de administración	/api/admin/*: news, members (+photo, photos), groups, events, projects, files, messages, content-blocks, page-visibility, intranet/users, intranet/files, accounts, translate

Tabla 9. Mapa de rutas de la aplicación.

B.4.3. Arquitectura de la versión inglesa

La versión inglesa no duplica el árbol de rutas: el middleware reescribe `/en/*` a la ruta española equivalente añadiendo la cabecera `x-locale: en`, que los Server Components leen a través de un helper de servidor; los componentes de cliente reciben el idioma como propiedad. Los helpers de cliente construyen los enlaces internos con el prefijo correcto, de modo que navegar por `/en` se mantiene siempre en inglés. El panel, el área de miembros y las API no tienen versión inglesa: sus rutas `/en/*` redirigen a la española.

Decisión relevante: el conmutador ES|EN de la cabecera es un enlace de documento completo (`<a>` nativo), no un enlace de cliente de Next. Como ambos idiomas comparten el mismo árbol de rutas interno, la navegación de cliente reutilizaría la página ya renderizada en el otro idioma (se comprobó durante el desarrollo: cambiaba la URL pero no el contenido); la navegación completa vacía la caché del router y garantiza la coherencia. Cualquier enlace futuro que cruce idiomas debe seguir la misma regla.

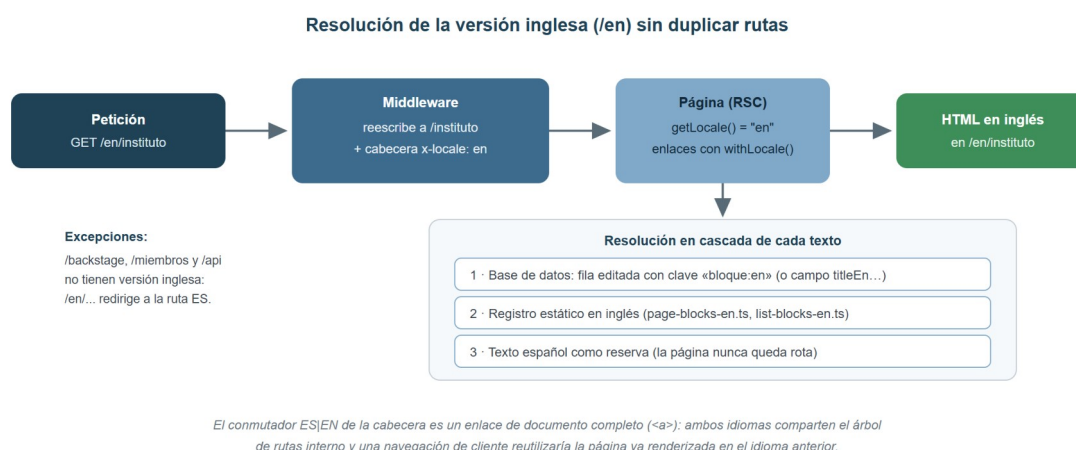


Figura 5. Resolución de la versión inglesa: reescritura del middleware y cascada de contenido.

B.4.4. Presentación y experiencia de usuario

- Identidad visual propia sobre tokens de marca (variables CSS consumidas por Tailwind), derivada de los prototipos de la fase de diseño (docs/design).

- Tema claro y oscuro en el sitio público: la preferencia se guarda en el navegador y se aplica sin parpadeo mediante un script previo al renderizado; las gráficas de Estadísticas se recolorean en vivo al cambiar de tema. El panel es solo claro.
- Editor TipTap con barra de formato, tablas, enlaces e imágenes que se suben directamente a la biblioteca de archivos.
- Accesibilidad: enlace de salto al contenido, navegación por teclado, textos alternativos y animaciones que respetan la preferencia de movimiento reducido; los tokens de color definen ambos temas de forma consistente.
- Correo institucional con plantillas HTML coherentes con la web (cabecera con logo incrustado como adjunto en línea, botón de acción, versión de texto plano).

B.4.5. Seguridad

- Autenticación del panel con NextAuth v5 (Credentials): contraseñas con hash bcrypt y sesión JWT con rol. El middleware exige rol de administración en /backstage/** y /api/admin/** (401/403 en API, redirección al login en páginas).
- El área de miembros usa un proveedor separado cuya sesión lleva el rol INTRANET: da acceso al área y a sus ficheros, nunca al panel.
- Tokens de acceso de un solo uso con caducidad de 30 minutos, consumidos al primer uso.
- Limitación de tasa en memoria (lib/rate-limit.ts) sobre la solicitud de enlaces, el formulario de contacto y el inicio de sesión del panel.
- Cabeceras de seguridad globales (nosniff, X-Frame-Options SAMEORIGIN, Referrer-Policy estricta, Permissions-Policy sin sensores) configuradas en next.config.js.
- Los documentos internos viven fuera de public/ y se sirven únicamente a sesiones válidas; robots.txt excluye el área privada.
- Los formularios validan en servidor con Zod; la entrada de usuario se escapa en las plantillas de correo.

B.4.6. Rendimiento

- Renderizado en servidor por defecto: la mayor parte del sitio llega como HTML, con JavaScript de cliente solo donde hay interactividad.
- Imágenes optimizadas por el pipeline de Next (tamaños acotados a los realmente migrados) y fotografías de miembros normalizadas a 512 px con sharp en el momento de subirlas.
- El layout público se marca como dinámico deliberadamente para que menú y visibilidad de páginas reaccionen al instante a los cambios del panel; las gráficas de Estadísticas se montan al entrar en el viewport.
- Consultas Prisma tipadas con restricciones de unicidad en los puntos calientes (slug de noticia, par página+clave de bloque, correo de usuario).

B.4.7. Despliegue

En desarrollo, docker compose levanta únicamente PostgreSQL (puerto 5433) y la aplicación corre con el servidor de desarrollo de Next; el entorno se reconstruye con la semilla y los scripts de datos. En producción, el modelo es el ya rodado con IUCE Reservas en el CPD de la USAL: la aplicación y PostgreSQL como contenedores Docker orquestados con docker-compose, tras Apache 2 como proxy inverso con certificado Let's Encrypt. Tres conjuntos de datos viven fuera del repositorio y forman parte de la copia de seguridad del despliegue: la base de datos, las imágenes subidas y migradas (public/uploads) y los documentos internos (storage/). Las variables de entorno relevantes (base de datos, secreto de sesión, URL pública, claves de Resend y DeepL, credenciales iniciales) se documentan en el Anexo C.